Practica repositorio con Github

Bruce Derek Peña Contreras
Compañera: Carol Torres
Ficha: 3406012
Instructora: Isaura Suarez

Resumen

En la presente práctica se implementó un flujo de trabajo colaborativo utilizando Git y GitHub entre dos desarrolladores. El proceso abarcó la creación de un repositorio remoto, su clonación en entornos locales, la creación de ramas independientes para la construcción de características y la posterior fusión de cambios hacia la rama principal (`main`). Durante el proceso de integración, se generó un conflicto de versiones debido a la modificación simultánea de un mismo archivo en ambas ramas. El conflicto fue identificado y resuelto en la plataforma, consolidando la versión final en la rama principal.

Introducción y Objetivos

Marco Teórico

Git es un sistema de control de versiones distribuido que permite rastrear cambios en el código fuente. GitHub actúa como plataforma de alojamiento remoto para facilitar la colaboración. Cuando dos o más desarrolladores modifican las mismas líneas de un archivo de manera concurrente en ramas distintas, Git detiene el proceso automático de fusión (*merge*) y requiere que los desarrolladores intervengan manualmente para seleccionar la versión definitiva, garantizando así la integridad del código.

Objetivos

- **Objetivo General:** Aplicar el flujo de trabajo colaborativo en Git y GitHub identificando y resolviendo conflictos de fusión de código.
- **Objetivos Específicos:**

1. Crear y clonar un repositorio remoto de GitHub en el entorno local.
2. Implementar el trabajo en ramas secundarias para aislar los cambios de cada desarrollador.
3. Ejecutar la fusión de ramas hacia `main` y resolver los conflictos de versiones generados por ediciones concurrentes.

Herramientas y Entorno

- **Sistema de Control de Versiones:** Git (CLI / Terminal)
- **Plataforma de Alojamiento Remoto:** GitHub
- **Editor de Código:** Visual Studio Code / Editor de texto
- **Equipos de cómputo:** 2 estaciones de trabajo locales

Procedimiento

1. Creación e Inicialización del Repositorio:

- Se creó un repositorio remoto en la plataforma GitHub.
- Ambos integrantes ejecutaron la clonación del repositorio en sus respectivos ordenadores mediante el comando:

git clone
https://github.com/Bruce-coder17/Practica-en-clase-15-10-2026.git

2. Desarrollo en Ramas Independientes:

- Cada integrante creó y se desplazó a una rama de trabajo propia para aislar los cambios:
- Bash

git checkout -b <nombre-de-la-rama>

- Cada integrante creó y se desplazó a una rama de trabajo propia para aislar los cambios: }

  Carol: git checkout -b practica-carol

  Bruce: git checkout -b practica-bruce

- Se realizaron modificaciones sobre el archivo de trabajo designado de forma simultánea pero independiente.

3. Confirmación y Envío de Cambios:

- Se guardaron y confirmaron los cambios en los repositorios locales:

  git add .

  git commit -m "Cree una función de Python que permite: Registrar libros, mostrar todos los libros, buscar un libro, prestar un libro y devolver un libro"

- Se enviaron las ramas al repositorio remoto en GitHub:

  git push origin practica-bruce

4. Fusión y Generación del Conflicto:

- El primer integrante completó la fusión de su rama con la rama `main` exitosamente en GitHub.
- Cuando el segundo integrante intentó fusionar su rama a `main`, Git detectó discrepancias e impidió la fusión automática, notificando un **conflicto de versiones** (*merge conflict*).

5. Resolución del Conflicto:

- Se inspeccionaron las diferencias entre ambas ramas directamente en la interfaz de GitHub / editor local.
- Se seleccionó la versión propuesta por el primer integrante para prevalecer sobre la rama principal.
- Se confirmó la resolución del conflicto y se completó la fusión definitiva hacia `main`.

Resultados

- **Estado Inicial:** Repositorio remoto sincronizado con dos ramas de desarrollo paralelas partiendo del mismo commit base.

- **Conflicto Detectado:** Impase durante la integración de la segunda rama a la rama `main` debido a cambios en las mismas líneas del archivo.
- **Estado Final:** Rama `main` actualizada con la versión seleccionada. El historial de commits del repositorio refleja el proceso de bifurcación, la primera integración y el commit de resolución de conflicto.

Discusión de Resultados

El conflicto surgido durante la práctica es un comportamiento esperado en entornos colaborativos cuando no existe una sincronización previa antes de la fusión. Al elegir únicamente una de las dos versiones, se logró desbloquear la rama principal y consolidar la versión elegida, aunque esta acción provocó que los cambios de la segunda rama no quedarán reflejados en `main`.

Para optimizar este proceso en futuras prácticas, se identificó la importancia de ejecutar `git pull origin main` en la rama local antes de intentar la fusión, además de evaluar la integración combinada de ambos códigos en lugar de descartar por completo una de las versiones.

Conclusiones

1. Se comprendió el funcionamiento práctico de las ramas en Git como mecanismo para aislar tareas y evitar la alteración directa de la rama de producción (`main`).
2. Se comprobó que Git no pierde información automáticamente, sino que detiene el flujo para solicitar la intervención humana cuando existen ediciones contradictorias.
3. Se determinó que la comunicación constante y la actualización frecuente desde la rama principal son prácticas fundamentales para minimizar los conflictos de integración en proyectos de desarrollo de software.